

Data-Driven Algorithms

By exploring a spell checker program, students discover that AI relies on *data-driven algorithms*. When we provide more representative data to the computer, data-driven algorithms generally produce a higher quality output.

Lesson Goals	Students will be able to... • Understand that machines and people often use starkly different processes to complete similar tasks, based on the strengths and limitations of computers and human brains. • Define data-driven algorithms as algorithms that produce better results when given more representative data. • Define values of various types. • Simplify a complex expression by replacing repeated parts with defined names, • Explain why variables are useful in math and programming.
Student-facing Lesson Goals	• Let's play with spell checkers to learn about <i>data-driven algorithms</i>.
Prerequisites	• Introduction to Artificial Intelligence
Materials	• PDF of all Handouts and Pages 5-6 • Spell Checker Starter File • Lesson Glossary • Lesson Slides • Printable Lesson Plan (a PDF of this web page)

Spell-checking: The Algorithm

Overview

Students consider one specific example of a data-driven algorithm: evaluating how spell checkers' programmed behaviors compare and contrast with the behaviors of humans.

Launch



Consider this sentence with your partner:

Teh cat meowed so loudky it woke thm upp.

- What do you think the author intended to write?
- Be prepared to describe how you did your spell checking:
 - How do you locate misspelled words?
 - How do you correct misspelled words?
 - How would you describe each of the spelling mistakes that the author made?



- How did you find the misspelled words?
 - *Answers will vary, but most people would say "I just knew it when I saw it". This is important to call out!*
- When you found a misspelled word, how did you figure out how to correct it?
 - *Possible responses might include things like:*
 - *There was only one word it could be!*
 - *I could tell from the rest of the sentence.*
 - *I've made this same typing mistake a million times.*
- The author made four different kinds of mistakes when writing this sentence. What were they?
 - **Teh** Reversed the letters **h** and **e**
 - **loudky** Typed **k** instead of **l**
 - **thm** Left out the letter **e**
 - **upp** Added an extra **p** at the end

When humans spell check, they rely on *context clues, experience, and background*. The process of spell-checking varies not just by person, but by situation. If a complex, technical word had been misspelled, for example, you might have relied on a different spell checking strategy.

Investigate

The spell checker built into your web browser is a classic example of a **data-driven algorithm** at work. To get a better sense of how machines work differently from us, we are going to follow the algorithm from the original SPELL program.



- To complete [The First Spell Checker](#), you will be the computer, following an early spell checker's algorithm.
- Reflect on the activity and how it compares to your own strategies.

After students have completed the activity, invite them to share their responses to the three reflection questions (below) in small groups.



- What do you think are some limitations of the first spell checker's algorithm?
 - *There is an extremely limited number of ways in which a word can be misspelled. For example, if two separate letters in a word are incorrect, the spell checker has no chance of proposing the correct word!*
- How is the first spell checker's algorithm similar to whatever strategy **you** use when spell checking your writing?
 - *See what your students propose, and discuss. If they come up with any similarities, we'd love to hear what they are!*
- How is the first spell checker's algorithm different from whatever strategy **you** use when spell checking your writing?
 - *When you check spelling, you rely a lot on context, instinct, and previously acquired knowledge.*
 - *The first spell checking program did not have, or use, any of these strengths.*
 - *In contrast, however, a human would not have the time, ability, or concentration to systematically list out thousands of potential words, whereas the computer has no difficulty doing so.*
 - *Therefore, the strategies used by the human and computer may be extremely different for a similar task.*

Synthesize



- In what ways do spell checkers succeed at mimicking humans?
 - *They don't! But the outcomes might be similar, depending on a wide variety of factors.*
- Would it be a good idea for a human to try mimic a spell checker's algorithm? Why or why not?
 - *No. it would be time-consuming, laborious, and highly error prone for a human.*
 - *The program we just discussed relies on computational brute force: primitive spell checking programs will repeatedly edit / transform a misspelled word, generating dozens of different words to verify in the dictionary.*

Spell Checking in Pyret

Overview

Students explore both the algorithm and the datasets that power a Pyret-based spell checker, discovering that **data-driven algorithms** are at the heart of AI.

Launch



- Let's take a look at a spell checking program written in Pyret.
- Open the [Spell Checker Starter File](#), save a copy, and click "Run".
- In Pyret, a *comment* is any line of code that starts with a `#`. Pyret ignores comments completely, so programmers use them to write notes! **Read the comments in the Definitions Area.** What do these notes tell you?
- Complete the first section of [A Pyret Spell Checker](#).

Heads up: The functions used in this Pyret starter file only work in this particular starter file.



- How did the spell checker's work compare to your efforts to be a computer on [The First Spell Checker](#)?
 - *It found everything we did and more than a hundred other insertions and substitutions in mere seconds!*
- Were all of Pyret's suggestions useful?
 - *No! Most of the words it proposed were nonsense.*

All of the "words" our spell checker generated in the first section involved a single change. A more technical term with the same meaning is **edit distance** - the edit distance for all of those words was 1, meaning they were a single swap, insertion, substitution or deletion away from the original.

Edit distance is one way that we can measure the similarity of two strings!



- Let's take a look at how Pyret generates lists of words with a bigger edit distance and refines its suggestions to remove nonsensical words.
- Complete the rest of [A Pyret Spell Checker](#).

- What did you discover about the dictionaries in [Spell Checker Starter File](#)?



- Why did `alt-words` return a different number of results, depending on which dictionary it was given?
 - *Because smaller dictionaries didn't know as many words, so `only-real` was removing words even though they could be found in larger dictionaries.*

When we fed *more representative data* to our spell checker, we got better results *without changing the spell checker's code*.

The effectiveness of ***data-driven algorithms*** is directly tied to how *representative* the data is.

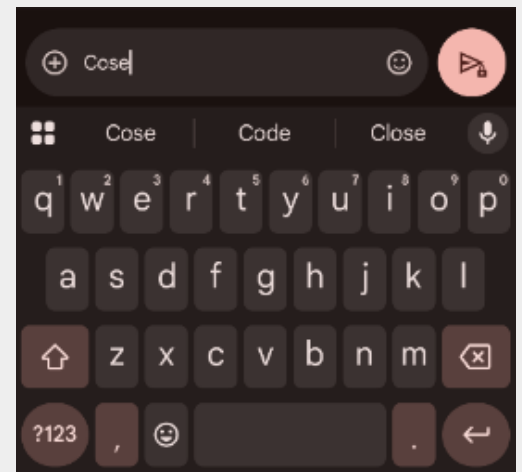
The same algorithm that almost seems "intelligent" when given 10,000 of the most commonly used words in English could appear *useless* with a dataset of only 100 words, or the names of 10,000 chemicals.

Synthesize



Look at the screenshot (right) of a text messaging app's suggestions for the possibly misspelled word "Cose". The app suggests the original word ("Cose"), indicating that no error was made, along with two alternatives: "Code" and "Close".

- What sort of *algorithm* do you think the app used in developing possible alternative words?
 - *Responses will vary.*
 - *Students may refer to the algorithms discussed in the first half of the lesson.*
 - *Students might also imagine more complicated algorithms - for instance, algorithms that consider the proximity of letters on the keyboard!*
- What sort of *data* do you think the spell checking app used in developing possible alternative words?
 - *Responses will vary.*
 - *Which words the user is most likely to type*
 - *The topic of the text conversation and the most probable next word*
 - *The dictionary of words the app draws from*



Students will discuss a similar screenshot of a text messaging app in our lesson on [Generative Models](#). During that lesson, however, students explore how generative AI uses data-driven algorithms to determine what word to produce next.

Defining Values

Overview

Students are introduced to variable definition in Pyret, motivated by a desire to save computationally-expensive operations to re-use their results.

Launch



Let's experiment with a bigger list of suggestions...

- In the Interactions Window on the right, evaluate `alt-words("plation", WORDS-L, 4)`.
 - How long did Pyret take to generate all the substitutions, deletions, insertions, and swaps, and then filter by only real words?
→ *A while!*
 - Would asking it to do the *same thing* over again go faster? Why or why not?
→ *Responses may vary*
 - Try it out! What happened? Can you explain why?
→ *It still took forever, because we are asking it to redo the same work.*

When we ask Pyret to do something over and over, it has no way of knowing that it should save its work! As a result, *it does the work over and over, too!* Fortunately, the world of math has a solution: define a **variable**.



Evaluate `results = alt-words("plation", WORDS-L, 4)`.

- Did anything come back? Why or why not?
 - *No! Defining a variable doesn't produce anything, **just like in math!***
 - *$x = 4$ doesn't have an "answer", because definitions aren't expressions.*
- Now evaluate `results` three times. What happens?
→ *It was lightning fast!*
- Click "Run", and evaluate `results`. What happens?
→ *We get an error message! Pyret "forgot" about our variable*
- Evaluate `feeling = "Joy"` and hit Enter. What do you think will happen if we evaluate `feeling`?

→ We'll get back "Joy"

When we want Pyret to remember a definition, we add it to the **Definitions Area**. Every time we click "Run" Pyret re-runs all the code, including the definition!



- Add `results = alt-words("plation", WORDS-L, 4)` as the **last line of the Definitions Area**, and click "Run". Evaluate `results` in the Interactions Area.
- What happens if you add another definition that *also* has the name `results`?



- Why are variable definitions valuable?
 - *We can save expensive work*
- What kinds of values can we define, besides just tables of spelling corrections?
 - *Anything!*
- Why can't we define two variables with the same name?
 - *Pyret only allows a variable name to have one value.*
- Now that we can generate and save spelling suggestions, what are some things we might want to *do* with them?
 - *We might want to sort the results, putting the suggestions with shorter edit distances first*
 - *We might want to just show the "top 5", rather than a table with hundreds of entries*
 - *Maybe we could make a chart or a graph?*